

Практическая работа №9

по дисциплине «Разработка веб-приложений на ASP.NET»

Реализация Web API с применением паттерна DTO и полным набором CRUD-операций

1. Цель работы

Реализовать Web API для существующей базы данных SneakerShop с использованием паттерна DTO. Контроллер не должен принимать и возвращать сущности Entity Framework напрямую. Необходимо реализовать операции Create, Read, Update, Delete и Patch с корректными HTTP-кодами 200, 201, 204 и 404.

2. Исходные данные

Новая база данных не создавалась. Использована база **SneakerShop** из скрипта BDASP.sql предыдущего семестра. Строка подключения указывает на SQL Server DESKTOP-RR3HEGD\SQLEXPRESS, каталог SneakerShop.

Основная сущность API — **Product** (таблица Products): ProductId, ProductName, Price, Stock, CategoryId. Контекст данных — существующий **SneakerShopContext**, он внедряется в контроллер через конструктор (Dependency Injection).

Поле CreatedAt требовалось заданием для возвращаемого DTO и операции POST. Оно добавлено в уже существующую таблицу Products при старте приложения (ALTER TABLE), без создания новой базы.

3. Классы DTO

Entity не используется в параметрах и возвращаемых значениях контроллера. Созданы отдельные классы:

- **ProductDto** — данные для клиента: все поля модели, включая ProductId и CreatedAt;
- **CreateProductDto** — тело POST: все поля, кроме ProductId и CreatedAt (их формирует сервер);

- **UpdateProductDto** — тело PUT: все поля обязательные;
- **PatchProductDto** — тело PATCH: все поля nullable (string?, int?, decimal?), передаются только изменяемые значения.

Статический маппер `ProductMapper.ToDto(Product product)` преобразует сущность в `ProductDto`.

4. Контроллер `ProductsController`

Маршрут `api/products`. Контекст `SneakerShopContext` внедряется через конструктор. Реализованы эндпоинты:

- GET `/api/products` — список с пагинацией (`page = 1`, `pageSize = 10` по умолчанию), сортировка по `ProductId`, код 200 OK;
- GET `/api/products/{id}` — одна запись в виде `ProductDto` или 404 Not Found;
- POST `/api/products` — создание, `CreatedAt = DateTime.UtcNow`, `SaveChanges()`, 201 Created через `CreatedAtAction(GetById)`;
- PUT `/api/products/{id}` — полное обновление всех полей, 204 No Content или 404;
- PATCH `/api/products/{id}` — частичное обновление полей, которые в DTO не равны null, 204 или 404;
- DELETE `/api/products/{id}` — удаление, `SaveChanges()`, 204 или 404.

5. Результаты тестирования

Проверка выполнена через Swagger UI (<http://localhost:5014/swagger>) и HTTP-запросы к запущенному API. Использованы реальные данные таблицы `Products` базы `SneakerShop`.

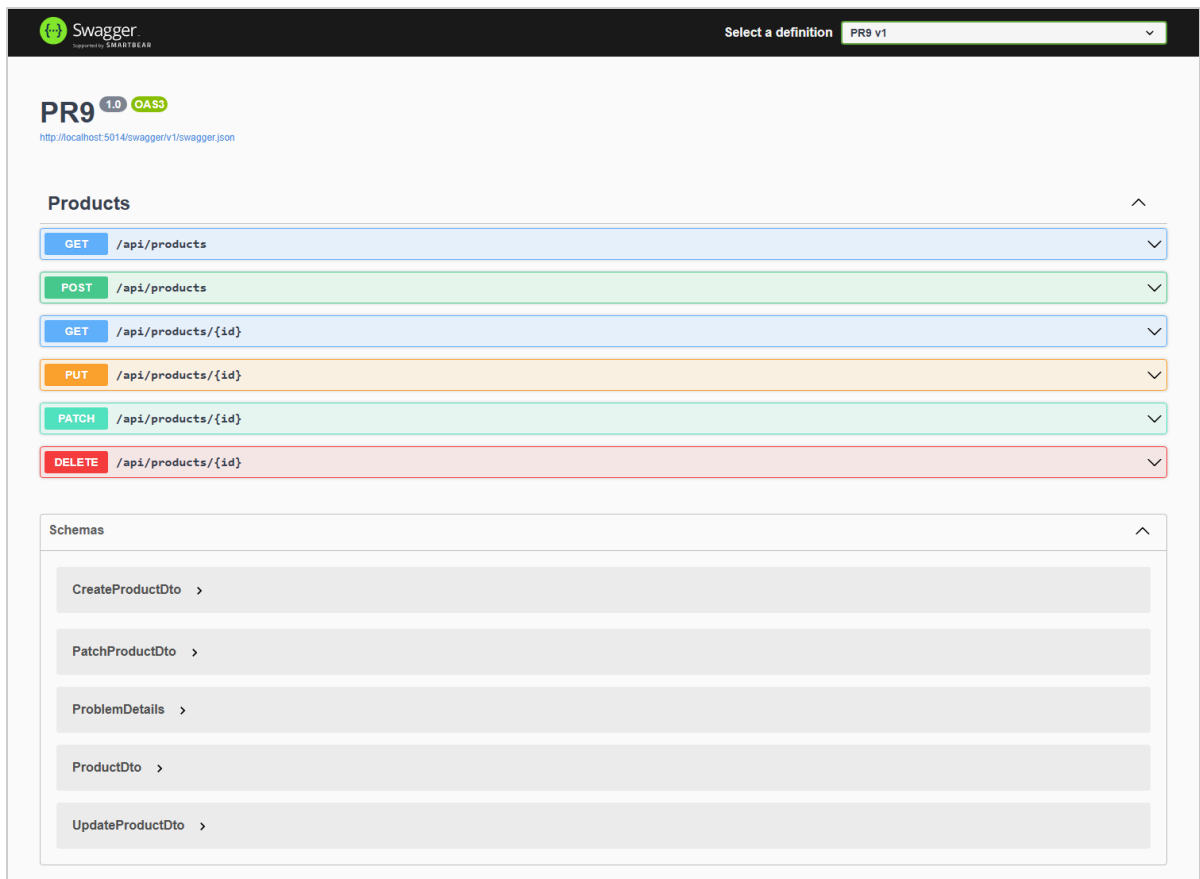


Рисунок 1. Swagger UI: набор CRUD-методов и схемы DTO

Проверка методов Web API через HTTP-запросы

База данных SneakerShop · маршрут api/products · реальные ответы сервера http://localhost:5014

GET /api/products?page=1&pageSize=3200 OK	
ЗАПРОС	ОТВЕТ
<pre>page = 1 pageSize = 3 Сортировка: ProductId</pre>	<pre>{ { "productId": 1, "productName": "Nike Air Max", "price": 120.00, "stock": 10, "categoryId": 1, "createdAt": "2026-09-15T00:08:59.825338" }, { "productId": 2, "productName": "Adidas Ultraboost", "price": 11990.00, "stock": 5, "categoryId": 4, "createdAt": "2026-09-15T00:08:59.825338" }, { "productId": 3, "productName": "Air Jordan 1", "price": 12990.00, "stock": 7, "categoryId": 5, "createdAt": "2026-09-15T00:08:59.825338" } }</pre>
GET /api/products/1200 OK	
ЗАПРОС	ОТВЕТ PRODUCTDTO
<pre>id = 1</pre>	<pre>{ "productId": 1, "productName": "Nike Air Max", "price": 120.00, "stock": 10, "categoryId": 1, "createdAt": "2026-09-15T00:08:59.825338" }</pre>
GET /api/products/9999404 Not Found	
ЗАПРОС	ОТВЕТ
<pre>id = 9999 (запись отсутствует)</pre>	<pre>404 Not Found</pre>
POST /api/products201 Created	
ТЕЛО CREATEPRODUCTDTO	ОТВЕТ PRODUCTDTO
<pre>{ "productName": "Asics Gel-Kayano", "price": 190.00, "stock": 7, "categoryId": 4 }</pre>	<pre>Location: /api/products/7 { "productId": 7, "productName": "Asics Gel-Kayano", "price": 190.00, "stock": 7, "categoryId": 4, "createdAt": "2026-09-15T00:09:19.8892592" }</pre>
PUT /api/products/7204 No Content	
ТЕЛО UPDATEPRODUCTDTO	ОТВЕТ
<pre>{ "productName": "Asics Gel-Kayano 31", "price": 199.99, "stock": 5, "categoryId": 4 }</pre>	<pre>204 No Content Тело ответа отсутствует</pre>
PATCH /api/products/7204 No Content	
ТЕЛО PATCHPRODUCTDTO	ОТВЕТ
<pre>{ "stock": 3 }</pre>	<pre>204 No Content Обновлено только поле stock</pre>
GET /api/products/7200 OK	
ЗАПРОС	ОТВЕТ
<pre>Проверка после PUT и PATCH</pre>	<pre>{ "productId": 7, "productName": "Asics Gel-Kayano 31", "price": 199.99, "stock": 3, "categoryId": 4, "createdAt": "2026-09-15T00:09:19.889259" }</pre>

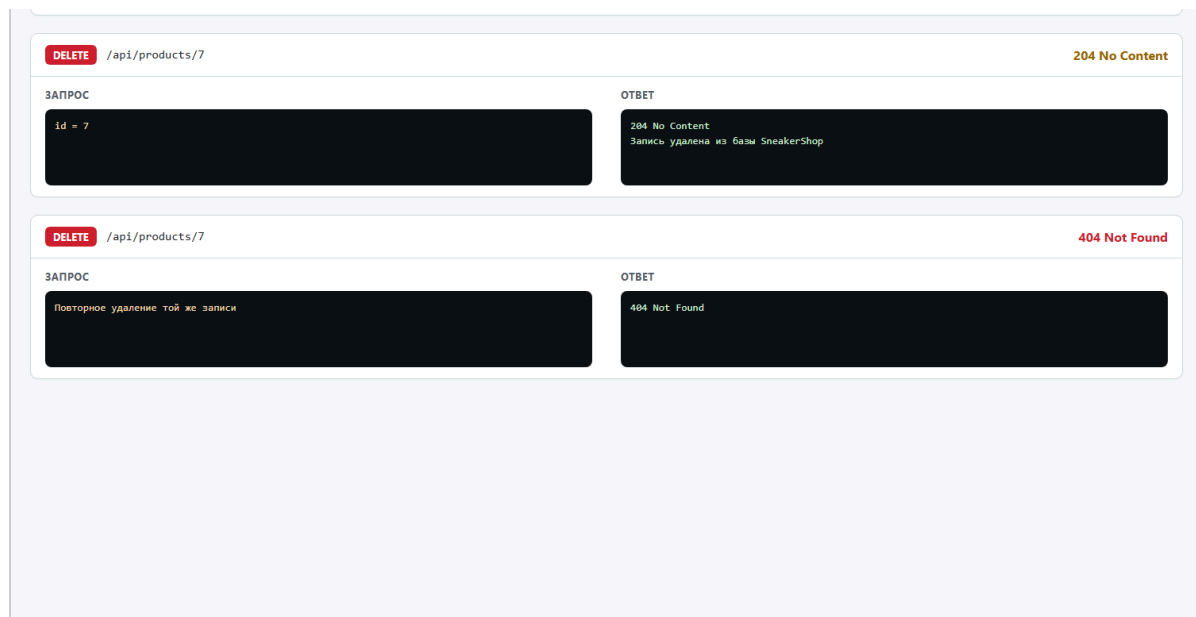


Рисунок 2. Запросы и ответы API: 200 OK, 201 Created, 204 No Content, 404 Not Found

6. Соответствие кодов ответа

- GET списка — 200 OK, возвращён массив `ProductDto`, сортировка по `Id`, пагинация `pageSize=3`.
- GET по `id=1` — 200 OK (Nike Air Max); GET `id=9999` — 404 Not Found.
- POST — 201 Created, заголовок `Location: /api/products/7`, `CreatedAt` задан сервером (UTC).
- PUT и PATCH — 204 No Content; последующий GET показал обновлённые имя, цену и остаток.
- DELETE — 204 No Content; повторный DELETE той же записи — 404 Not Found.

7. Вывод

Web API реализован на существующей базе `SneakerShop` и модели `Product`. Контроллер использует DTO и статический маппер, не отдаёт Entity клиенту. Реализованы все CRUD-операции, включая PATCH, пагинацию списка и коды 200, 201, 204, 404. Работа методов подтверждена через Swagger и прямые HTTP-запросы.